



EXAMEN PRÁCTICO: MANTENIMIENTO DE SOFTWARE

PROYECTO: CATÁLOGO DE
SMARTPHONES

CONSTRUCCIÓN Y EVOLUCIÓN
DE SOFTWARE

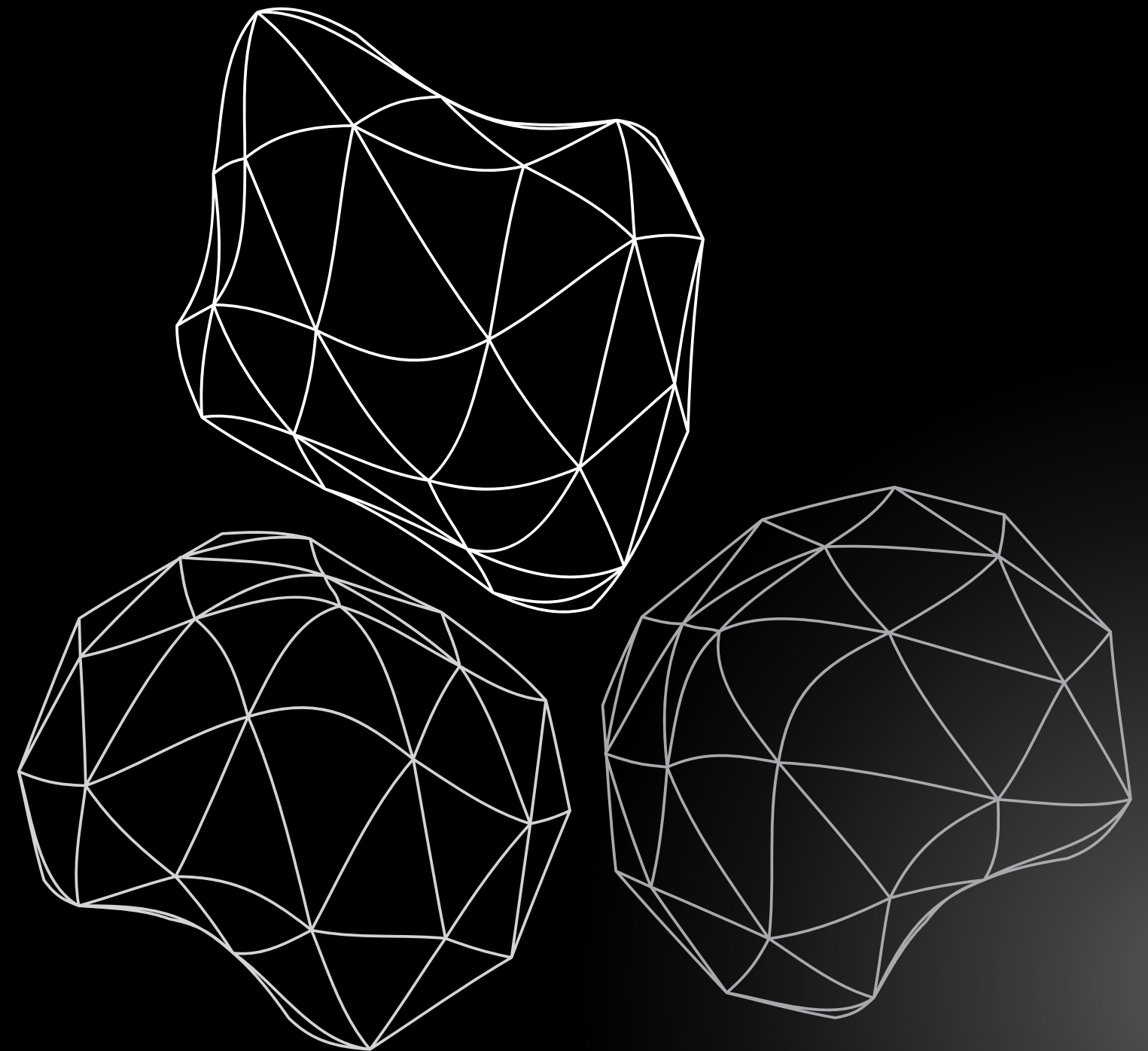
AARON CHAUCA

OBJETIVO GENERAL

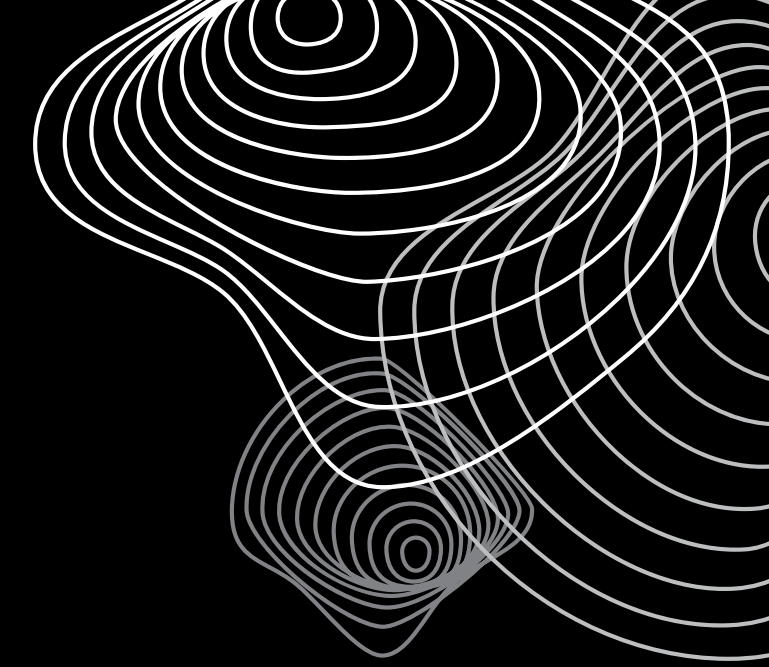
Aplicar los cuatro tipos de mantenimiento de software (correctivo, adaptativo, perfectivo y preventivo) sobre la carpeta “epn-event-manager” y en el CRUD del proyecto de catálogos de smartphones para mejorar su robustez, seguridad, mantenibilidad y calidad.

OBJETIVOS ESPECIFICOS

- Identificar deuda técnica.
- Corregir errores existentes.
- Prevenir problemas futuros.



MANTENIMIENTO CORRECTIVO



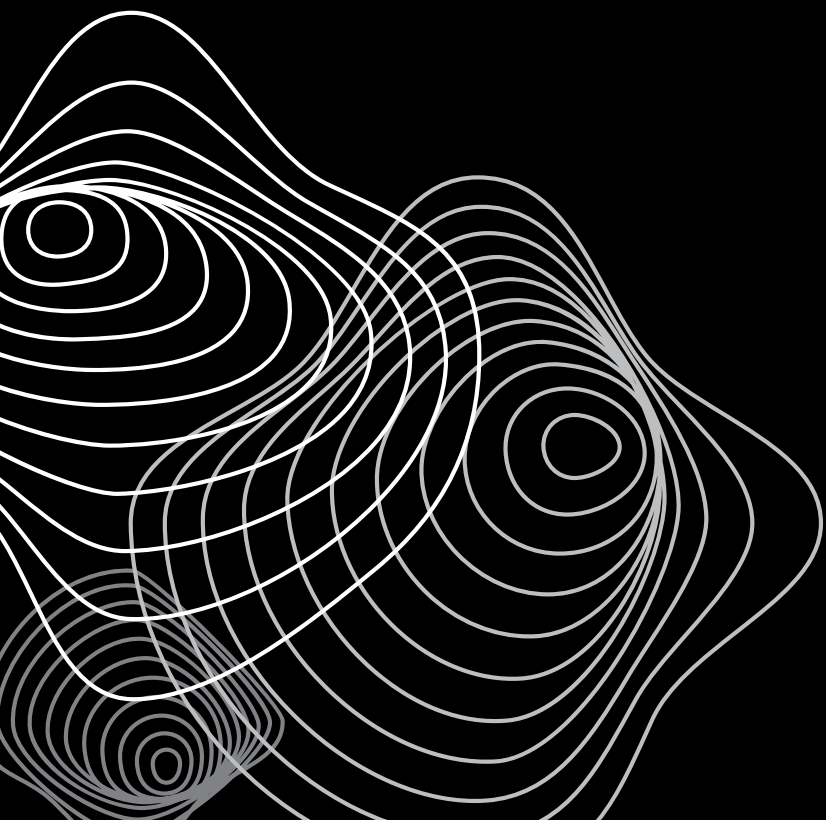
Problemas encontrados:

- Uso de console.log sin estructura.
- Excepciones no controladas.
- Falta de atomicidad en create smartphone.
- Eventos sin manejo de errores.
- Responses positivos sin persistencia real en la BD.
- Fechas almacenadas inconsistentemente

Cambios realizados:

- Logger estructurado.
- Try-catch.
- Manejo controlado de excepciones.
- Corrección del flujo create + registerEvent.
- Persistencia de eventos en Delete.
- Validaciones mediante "ParseIntPipe".

Este mantenimiento se clasifica como correctivo porque se enfocó en corregir defectos funcionales ya existentes que generaban comportamientos incorrectos y datos inconsistentes dentro del sistema.



MANTENIMIENTO CORRECTIVO - LOGS ESTRUCTURADOS CON NIVELES Y TIMESTAMP

Antes

```
console.log(`CREATE smartphone ${saved.id}`);
```

```
console.log('QUERY smartphones');
```

```
console.log(`QUERY smartphone ${id}`);
```

```
console.log(`DELETE smartphone ${id}`);
```

Se reemplazó console.log por el Logger de NestJS en todos los servicios, registrando nivel INFO en operaciones exitosas, WARN cuando un recurso no existe, y ERROR ante fallos de infraestructura, siempre con timestamp ISO 8601.

Después

```
@Injectable()
export class SmartphonesService {
  // Antes: console.log sin niveles ni timestamp - ahora Logger con INFO/WARN/ERROR e ISO 8601
  private readonly logger = new Logger(SmartphonesService.name);
```

```
export class EventsService {
  // Antes: sin Logger ni try-catch - ahora Logger con INFO/ERROR y errores controlados
  private readonly logger = new Logger(EventsService.name);
```


MANTENIMIENTO CORRECTIVO

Antes

```
async create(dto: CreateSmartphoneDto): Promise<SmartphoneEntity> {
  this.validateBasicFields(dto);

  const now = new Date().toISOString();
  const smartphone = this.smartphonesRepo.create({
    brand: dto.brand.trim(),
    model: dto.model.trim(),
    price: Number(dto.price),
    storage: dto.storage.trim(),
    created_at: now,
    updated_at: now,
  });

  const saved = await this.smartphonesRepo.save(smartphone);
  console.log(`CREATE smartphone ${saved.id}`);
  await this.registerCrudEvent(
    EventAction.Create,
    'Smartphone creado',
    saved,
  );
  return saved;
}

async findAll(): Promise<SmartphoneEntity[]> {
  const smartphones = await this.smartphonesRepo.find({
    order: { id: 'ASC' },
  });
  console.log('QUERY smartphones');
  await this.registerCrudEvent(EventAction.Query, 'Smartphones consultados', {
    total: smartphones.length,
  });
  return smartphones;
}

async findOne(id: number): Promise<SmartphoneEntity> {
  const smartphone = await this.smartphonesRepo.findOneBy({ id });
  if (!smartphone) {
    throw new NotFoundException('Smartphone no encontrado');
  }
}
```

Se agregaron bloques try-catch en todos los métodos de los servicios para que un fallo de base de datos devuelva un error 500 controlado al cliente en lugar de propagar una excepción no manejada que tumbaría el servidor.

Después

```
async create(dto: CreateSmartphoneDto): Promise<SmartphoneEntity> {
  this.validateBasicFields(dto);
  // Antes: sin try-catch - ahora errores de BD quedan controlados
  try {
    const now = new Date().toISOString();
    const smartphone = this.smartphonesRepo.create({
      brand: dto.brand.trim(),
      model: dto.model.trim(),
      price: Number(dto.price),
      storage: dto.storage.trim(),
      created_at: now,
      updated_at: now,
    });

    const saved = await this.smartphonesRepo.save(smartphone);
    this.logger.log({
      operation: 'CREATE',
      entity: 'smartphone',
      id: saved.id,
      result: 'ok',
      timestamp: new Date().toISOString(),
    });
    await this.safeRegisterEvent(EventAction.Create, 'Smartphone creado', saved);
    return saved;
  } catch (error) {
    if (error instanceof BadRequestException || error instanceof NotFoundException) {
      throw error;
    }
    this.logger.error({
      operation: 'CREATE',
      entity: 'smartphone',
      result: 'error',
      message: (error as Error).message,
      timestamp: new Date().toISOString(),
    });
    throw new InternalServerErrorException('Error al crear el smartphone');
  }
}
```

MANTENIMIENTO CORRECTIVO - MANEJO DE ERRORES DE BASE DE DATOS

Antes

```
const saved = await this.smartphonesRepo.save(smartphone);
console.log(`CREATE smartphone ${saved.id}`);
await this.registerCrudEvent(
  EventAction.Create,
  'Smartphone creado',
  saved,
);
```

Se agregaron bloques try-catch en todos los métodos de los servicios para que un fallo de base de datos devuelva un error 500 controlado al cliente en lugar de propagar una excepción no manejada que tumbaría el servidor.

Después

```
async create(dto: CreateSmartphoneDto): Promise<SmartphoneEntity> {
  this.validateBasicFields(dto);
  // Antes: sin try-catch - ahora errores de BD quedan controlados
  try {
    const now = new Date().toISOString();
    const smartphone = this.smartphonesRepo.create({
      brand: dto.brand.trim(),
      model: dto.model.trim(),
      price: Number(dto.price),
      storage: dto.storage.trim(),
      created_at: now,
      updated_at: now,
    });

    const saved = await this.smartphonesRepo.save(smartphone);
    this.logger.log({
      operation: 'CREATE',
      entity: 'smartphone',
      id: saved.id,
      result: 'ok',
      timestamp: new Date().toISOString(),
    });
    await this.safeRegisterEvent(EventAction.Create, 'Smartphone creado', saved);
    return saved;
  } catch (error) {
    if (error instanceof BadRequestException || error instanceof NotFoundException) {
      throw error;
    }
    this.logger.error({
      operation: 'CREATE',
      entity: 'smartphone',
      result: 'error',
      message: (error as Error).message,
      timestamp: new Date().toISOString(),
    });
    throw new InternalServerErrorException('Error al crear el smartphone');
  }
}
```

MANTENIMIENTO CORRECTIVO - BUG DE PROPAGACIÓN EN REGISTRO DE EVENTOS

Antes

```
async registerEvent(dto: CreateEventDto): Promise<RegisterEventResult> {
  const payloadStr = JSON.stringify(dto.payload ?? {});
  const event = this.eventsRepo.create({
    source: dto.source,
    entity: dto.entity,
    action: dto.action,
    title: dto.title,
    description: dto.description ?? null,
    payload: payloadStr,
    created_at: new Date().toISOString(),
  });

  await this.eventsRepo.save(event);
  return { ok: true };
}

async findAll(): Promise<EventEntity[]> {
  return this.eventsRepo.find({
    order: { created_at: 'DESC', id: 'DESC' },
  });
}

async findBySource(source: string): Promise<EventEntity[]> {
  return this.eventsRepo.find({
    where: { source },
    order: { created_at: 'DESC', id: 'DESC' },
  });
}
```

Se corrigió el bug donde un fallo al guardar el evento de auditoría cancelaba la operación CRUD aunque el dato ya había sido persistido. Ahora el fallo del evento se registra como WARN sin afectar la respuesta al cliente.

Después

```
async registerEvent(dto: CreateEventDto): Promise<RegisterEventResult> {
  // Antes: sin try-catch - ahora error de persistencia controlado y logueado
  try {
    const payloadStr = JSON.stringify(dto.payload ?? {});
    const event = this.eventsRepo.create({
      source: dto.source,
      entity: dto.entity,
      action: dto.action,
      title: dto.title,
      description: dto.description ?? null,
      payload: payloadStr,
      created_at: new Date().toISOString(),
    });

    await this.eventsRepo.save(event);
    this.logger.log({
      operation: 'REGISTER_EVENT',
      action: dto.action,
      entity: dto.entity,
      result: 'ok',
      timestamp: new Date().toISOString(),
    });
    return { ok: true };
  } catch (error) {
    this.logger.error({
      operation: 'REGISTER_EVENT',
      action: dto.action,
      entity: dto.entity,
      result: 'error',
      message: (error as Error).message,
      timestamp: new Date().toISOString(),
    });
    throw new InternalServerErrorException('Error al registrar el evento');
  }
}
```


MANTENIMIENTO CORRECTIVO - EL EVENTO DELETE NO SE GUARDA, PERO RESPONDE ÉXITO

Antes

```
1  if (action === 'DELETE') {
2    // BUG INTENCIONAL (correctivo): se construye el objeto pero se devuelve
3    // exito antes de persistirlo. El save nunca se ejecuta.
4    this.deleteRepo.create({
5      source: dto.source,
6      entity: dto.entity,
7      action: dto.action,
8      title: dto.title,
9      payload: payloadStr,
10     createdAt: localDate,
11   });
12   return { ok: true }; // <-- jamás llega a .save()
13 }
```

Después

```
1  if (['CREATE', 'UPDATE', 'DELETE', 'QUERY'].includes(action)) {
2    // Extraemos el término de búsqueda si el payload lo contiene
3    const queryTerm = dto.payload?.query_term
4      ? String(dto.payload.query_term)
5      : '';
6
7    const ev = this.eventRepo.create({
8      source: dto.source,
9      entity: dto.entity,
10     action: dto.action,
11     title: dto.title,
12     description: dto.description,
13     payload: dto.payload as EventPayload, // Guardamos el payload como JSON estructurado
14     created_at: isoDate,
15     query_term: queryTerm,
16   });
17   await this.eventRepo.save(ev);
18   return { ok: true };
19 }
20
21 return { ok: false };
22 }
```

El sistema convertía manualmente los IDs usando Number(id), permitiendo valores inválidos como NaN y provocando errores silenciosos.

MANTENIMIENTO CORRECTIVO - FECHAS GUARDADAS COMO TEXTO LOCAL Y CON NOMBRES DISTINTOS

Antes

```
1 const localDate = new Date().toLocaleString();
```

```
1 // Inconsistencia: timestamp guardado como texto local, no UTC
2 @Column({ nullable: true })
3 timestamp: string;
```

Después

```
1 // Fecha guardada en formato ISO/UTC estandarizado
2 const isoDate = new Date().toISOString();
```

```
1 @Column({ type: 'datetime', default: () => 'CURRENT_TIMESTAMP' })
2 created_at!: string;
```

Cuatro columnas de fecha con nombres distintos y formato local hacían imposible un ordenamiento cronológico real. Al unificar a created_at con formato ISO 8601, el ordenamiento SQL funciona correctamente en cualquier entorno

MANTENIMIENTO CORRECTIVO - PARÁMETROS NUMÉRICOS SIN VALIDACIÓN

Antes



Después

```
1  @Put('/:id')
2  update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateTaskDto) {
3    return this.tasksService.update(id, dto);
4  }
5
6  @Delete('/:id')
7  remove(@Param('id', ParseIntPipe) id: number) {
8    return this.tasksService.remove(id);
9  }
```

ParseIntPipe rechaza la petición con 400 Bad Request si el :id no es un entero válido. Sin él, PUT /tasks/texto llega al servicio con id = NaN y la consulta TypeORM produce resultados indeterminados.

MANTENIMIENTO ADAPTATIVO

Problemas:

- Endpoints públicos.
- No existía API Key.
- Variables de entorno incompletas.
- La base SQLite estaba hardcodeada.
- El sistema depende de synchronize: true para crear automáticamente estructuras en la base de datos.

Solución:

- API Key Guard.
- ConfigModule.
- Archivo .env.
- Validación de variables al arranque.
- Migración de configuración hacia variables de entorno.
- El proyecto trabaja en función de `deDB_SYNCHRONIZE=false`

Se considera mantenimiento adaptativo porque el objetivo principal fue adaptar el sistema para que pueda ejecutarse correctamente en diferentes ambientes y configuraciones futuras.

MANTENIMIENTO ADAPTATIVO - GUARD DE API KEY EN SMARTPHONES

Antes

```
@Controller('api/smartphones')
export class SmartphonesController {
  constructor(private readonly smartphonesService: SmartphonesService) {}

  @Post()
  create(@Body() dto: CreateSmartphoneDto) {
    return this.smartphonesService.create(dto);
  }

  @Get()
  findAll() {
    return this.smartphonesService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.smartphonesService.findOne(Number(id));
  }

  @Put(':id')
  update(@Param('id') id: string, @Body() dto: UpdateSmartphoneDto) {
    return this.smartphonesService.update(Number(id), dto);
  }

  @Delete(':id')
  remove(@Param('id') id: string) {
    return this.smartphonesService.remove(Number(id));
  }
}
```

Se aplicó
ApiKeyGuard a los 5
endpoints del CRUD
de smartphones.
Cualquier solicitud
sin el header X-FIS-
EPN-KEY correcto
recibe un error 401.

Después

```
// Antes: endpoints públicos sin ninguna protección
// Ahora: todos los endpoints requieren header X-FIS-EPN-KEY via ApiKeyGuard
@ApiTags('Smartphones')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('api/smartphones')
export class SmartphonesController {
  constructor(private readonly smartphonesService: SmartphonesService) {}

  @Post()
  create(@Body() dto: CreateSmartphoneDto) {
    return this.smartphonesService.create(dto);
  }

  @Get()
  findAll() {
    return this.smartphonesService.findAll();
  }

  // Antes: Number(id) sin validación, NaN llegaba al service
  // Ahora: ParseIntPipe rechaza con 400 cualquier valor que no sea entero
  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.smartphonesService.findOne(id);
  }

  @Put(':id')
  update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateSmartphoneDto) {
    return this.smartphonesService.update(id, dto);
  }

  @Delete(':id')
  remove(@Param('id', ParseIntPipe) id: number) {
    return this.smartphonesService.remove(id);
  }
}
```

MANTENIMIENTO ADAPTATIVO - GUARD DE API KEY EN EVENTS

Antes

```
@Controller('events')
export class EventsController {
  constructor(private readonly eventsService: EventsService) {}

  @Post()
  registerEvent(@Body() dto: CreateEventDto) {
    return this.eventsService.registerEvent(dto);
  }

  @Get()
  findAll() {
    return this.eventsService.findAll();
  }

  @Get('source/:source')
  findBySource(@Param('source') source: string) {
    return this.eventsService.findBySource(source);
  }

  @Get('entity/:entity')
  findByEntity(@Param('entity') entity: string) {
    return this.eventsService.findByEntity(entity);
  }
}
```

Se protegieron los 4 endpoints del módulo de eventos con ApiKeyGuard.

Después

```
// Antes: endpoints de eventos públicos sin protección
// Ahora: todos requieren header X-FIS-EPN-KEY via ApiKeyGuard
@ApiTags('Events')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('events')
export class EventsController {
  constructor(private readonly eventsService: EventsService) {}

  @Post()
  registerEvent(@Body() dto: CreateEventDto) {
    return this.eventsService.registerEvent(dto);
  }

  @Get()
  findAll() {
    return this.eventsService.findAll();
  }

  @Get('source/:source')
  findBySource(@Param('source') source: string) {
    return this.eventsService.findBySource(source);
  }

  @Get('entity/:entity')
  findByEntity(@Param('entity') entity: string) {
    return this.eventsService.findByEntity(entity);
  }
}
```

MANTENIMIENTO ADAPTATIVO - GUARD DE API

KEY EN HEALTH Y STATS

Antes

```
@Controller('health')
export class HealthController {
  constructor(private readonly dataSource: DataSource) {}

  @Get()
  async check() {
    const timestamp = new Date().toISOString();

    try {
      await this.dataSource.query('SELECT 1');
      return { status: 'ok', database: 'ok', timestamp };
    } catch {
      throw new ServiceUnavailableException({
        status: 'error',
        database: 'down',
        timestamp,
      });
    }
  }
}
```

Se protegieron los endpoints de salud del sistema y estadísticas con ApiKeyGuard.

Después

```
// Antes: sin protección ni logs - ahora protegido con ApiKeyGuard y Logger estructurado
@ApiTags('Health')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('health')
export class HealthController {
  private readonly logger = new Logger(HealthController.name);

  constructor(private readonly dataSource: DataSource) {}

  @Get()
  async check() {
    const timestamp = new Date().toISOString();

    try {
      await this.dataSource.query('SELECT 1');
      this.logger.log({ operation: 'HEALTH_CHECK', result: 'ok', timestamp });
      return { status: 'ok', database: 'ok', timestamp };
    } catch {
      this.logger.error({
        operation: 'HEALTH_CHECK',
        result: 'error',
        database: 'down',
        timestamp,
      });
      throw new ServiceUnavailableException({
        status: 'error',
        database: 'down',
        timestamp,
      });
    }
  }
}
```


MANTENIMIENTO ADAPTATIVO - VARIABLES DE ENTORNO Y ARCHIVO .ENV

Antes

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'better-sqlite3',
      database: process.env.DB_PATH ?? 'db/events.sqlite',
      entities: [EventEntity, SmartphoneEntity],
      migrations: [
        CreateEventsTable1760000000000,
        CreateSmartphonesTable1760000001000,
      ],
      migrationsRun: process.env.DB_MIGRATIONS_RUN !== 'false',
      synchronize: process.env.DB_SYNCHRONIZE === 'true',
    }),
  ],
  exports: [TypeOrmModule],
})
export class DatabaseModule {}
```

Después

```
1  # Antes: variables de entorno no documentadas ni centralizadas
2  # Ahora: archivo .env con todas las variables requeridas por el sistema
3
4  PORT=3000
5  DB_PATH=db/events.sqlite
6  DB_MIGRATIONS_RUN=true
7  DB_SYNCHRONIZE=false
8
9  # Clave de acceso requerida en header X-FIS-EPN-KEY para todos los endpoints
10 API_KEY=epn-fis-2026-secret-key
11
```

Se creó el archivo .env con todas las variables del sistema (PORT, DB_PATH, API_KEY, entre otras), se añadió .env.example como plantilla pública y se excluyó .env del repositorio vía .gitignore.

MANTENIMIENTO ADAPTATIVO - ARRANQUE SEGURO Y DOCUMENTACIÓN API

Antes

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );
  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

Se mejoró el arranque de la aplicación agregando manejo limpio de apagado con `enableShutdownHooks()` para señales como `SIGTERM/SIGINT`, y se incorporó Swagger en `/api` para documentar y probar los endpoints del CRUD de smartphones con autenticación por API Key.

Después

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  // Antes: sin shutdown hooks – ahora maneja SIGTERM/SIGINT limpiamente al desplegar
  app.enableShutdownHooks();
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );

  // Antes: sin documentación de API – ahora Swagger disponible en /api
  const config = new DocumentBuilder()
    .setTitle('EPN Event Manager API')
    .setDescription('API CRUD de smartphones con registro de eventos')
    .setVersion('1.0')
    .addApiKey(
      { type: 'apiKey', name: 'x-fis-epn-key', in: 'header' },
      'x-fis-epn-key',
    )
    .build();
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);

  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

MANTENIMIENTO ADAPTATIVO - RUTA HARDCODEADA DE SQLITE Y EL SYNCHRONIZE: TRUE

Antes

```
TypeOrmModule.forRoot({  
  type: 'better-sqlite3',  
  database: 'db/events.sqlite',  
  entities: [  
    CreateEventEntity,  
    UpdateEventEntity,  
  ],  
  synchronize: true,  
}),
```

Se corrigió la configuración de SQLite eliminando valores hardcoded y trasladándolos a variables de entorno (.env). De esta forma, la ruta de la base de datos y la opción de sincronización se gestionan de manera dinámica, facilitando la configuración entre distintos entornos.

Después

```
.env  
1 DB_TYPE=better-sqlite3  
2 DB_NAME=db/events.sqlite  
3 DB_SYNCHRONIZE=false  
4
```

```
TypeOrmModule.forRootAsync({  
  imports: [ConfigModule],  
  inject: [ConfigService],  
  useFactory: (configService: ConfigService) => ({  
    type: configService.get<any>('DB_TYPE', 'better-sqlite3'),  
    database: configService.get<string>('DB_NAME', 'db/events.sqlite'),  
    entities: [TaskEntity, EventEntity],  
    synchronize: configService.get<string>('DB_SYNCHRONIZE') === 'true',  
  })),
```


MANTENIMIENTO PERFECTIVO

Problemas:

- Sin pruebas unitarias.
- Sin documentación API.
- Código plantilla innecesario.
- Cuatro tablas legacy para eventos con estructuras muy similares.
- Ordenamiento en memoria.
- Frontend estaba mezclado

Mejoras:

- Tests unitarios.
- Swagger.
- Eliminación de código muerto.
- Unificación del modelo de eventos mediante EventEntity y la tabla events.
- Separación de la vista HTML del controlador.

Este mantenimiento se clasifica como perfectivo porque se realizaron mejoras orientadas a optimizar la arquitectura, la mantenibilidad y la organización del sistema sin cambiar su funcionalidad principal.

MANTENIMIENTO PERFECTIVO - PRUEBAS UNITARIAS DE LÓGICA DE NEGOCIO

Antes

```
describe('AppController', () => {
  let appController: AppController;

  beforeEach(async () => {
    const app: TestingModule = await Test.createTestingModule({
      controllers: [AppController],
      providers: [AppService],
    }).compile();

    appController = app.get<AppController>(AppController);
  });

  describe('root', () => {
    it('should return "Hello World!"', () => {
      expect(appController.getHello()).toBe('Hello World!');
    });
  });
});
```

Después

```
const mockSmartphone: SmartphoneEntity = {
  id: 1,
  brand: 'Apple',
  model: 'iPhone 15',
  price: 999.99,
  storage: '128GB',
  created_at: '2026-05-29T00:00:00.000Z',
  updated_at: '2026-05-29T00:00:00.000Z',
};

const mockRepo = {
  create: jest.fn(),
  save: jest.fn(),
  find: jest.fn(),
  findOneBy: jest.fn(),
  remove: jest.fn()
}
```

Se crearon pruebas unitarias sobre SmartphonesService cubriendo los cinco métodos CRUD y los seis casos de validación de validateBasicFields (brand, model, price cero, price negativo, storage vacío).

MANTENIMIENTO PERFECTIVO - DOCUMENTACIÓN DE API CON SWAGGER

Antes

No existe
@nestjs/swagger
en el "package.json"

Se integró @nestjs/swagger y se configuró la interfaz disponible en /api con autenticación por API Key. Todos los DTOs fueron anotados con @ApiProperty para mostrar ejemplos de cada campo.

Después

```
"dependencies": {  
  "@nestjs/common": "^11.0.1",  
  "@nestjs/core": "^11.0.1",  
  "@nestjs/platform-express": "^11.0.1",  
  "@nestjs/swagger": "^11.4.4",  
  "@nestjs/typeorm": "^11.0.1",  
  "better-sqlite3": "^12.9.0",  
  "class-transformer": "^0.5.1",  
  "class-validator": "^0.15.1",  
  "reflect-metadata": "^0.2.2",  
  "rxjs": "^7.8.1",  
  "swagger-ui-express": "^5.0.1",  
  "typeorm": "^0.3.28"  
},
```


MANTENIMIENTO PERFECTIVO

Antes

```
@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

Se aplicó ApiKeyGuard a los endpoints del CRUD de smartphones. Cualquier solicitud sin el header X-FIS-EPN-KEY correcto recibe un error 401.

Después

```
// Antes: endpoints públicos sin ninguna protección
// Ahora: todos los endpoints requieren header X-FIS-EPN-KEY via ApiKeyGuard
@ApiTags('Smartphones')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('api/smartphones')
export class SmartphonesController {
  constructor(private readonly smartphonesService: SmartphonesService) {}

  @Post()
  create(@Body() dto: CreateSmartphoneDto) {
    return this.smartphonesService.create(dto);
  }

  @Get()
  findAll() {
    return this.smartphonesService.findAll();
  }
}
```

MANTENIMIENTO PERFECTIVO - MODELO DE EVENTOS

DIVIDIDO EN CUATRO TABLAS CASI IGUALES



Antes

```
1 @Entity('create_events')
2 export class CreateEventEntity {
3   @PrimaryGeneratedColumn()
4   id: number;
5
6   @Column({ nullable: true })
7   source: string;
8
9   @Column({ nullable: true })
10  entity: string;
11
12  @Column({ nullable: true })
13  action: string;
14
15  @Column({ nullable: true })
16  title: string;
17
18  @Column({ nullable: true })
19  description: string;
20
21  @Column({ type: 'text', nullable: true })
22  payload: string;
23
24  @Column({ nullable: true })
25  recorded_at: string;
26 }
27
```

Se consolidaron las entidades de eventos en una única EventEntity, eliminando repositorios y tablas redundantes. Además, se mejoró el manejo del payload mediante almacenamiento en formato JSON y se añadieron campos para facilitar la trazabilidad y consulta de eventos.

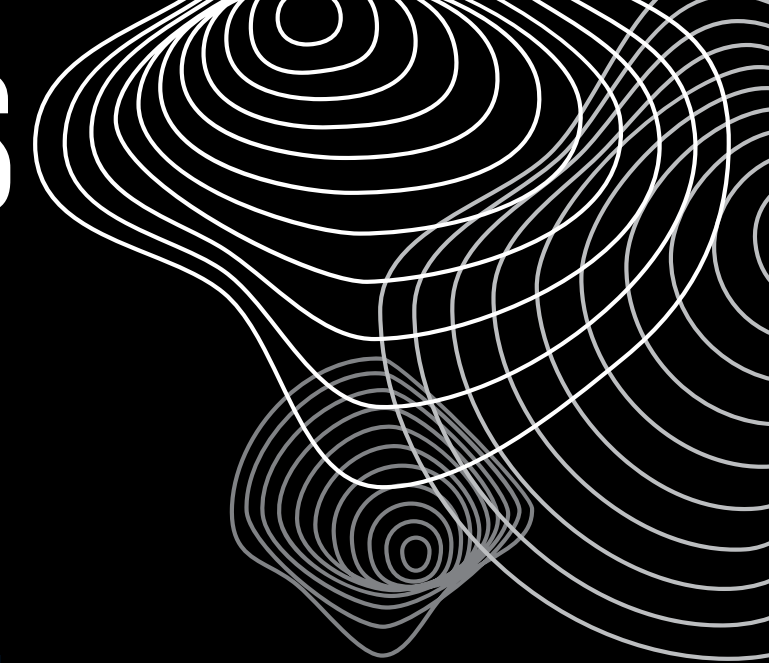
Después

```
1 @Entity('events')
2 export class EventEntity {
3   @PrimaryGeneratedColumn()
4   id!: number;
5
6   @Column({ nullable: true })
7   source!: string;
8
9   @Column({ nullable: true })
10  entity!: string;
11
12  @Column({ nullable: true })
13  action!: string;
14
15  @Column({ nullable: true })
16  title!: string;
17
18  @Column({ nullable: true })
19  description!: string;
20
21  @Column({ type: 'simple-json', nullable: true })
22  payload!: EventPayload;
23
24  @Column({ nullable: true })
25  query_term!: string;
26
27  @Column({ type: 'datetime', default: () => 'CURRENT_TIMESTAMP' })
28  created_at!: string;
29 }
```

```
1 constructor(
2   @InjectRepository(CreateEventEntity)
3   private readonly createRepo: Repository<CreateEventEntity>,
4   @InjectRepository(UpdateEventEntity)
5   private readonly updateRepo: Repository<UpdateEventEntity>,
6   @InjectRepository(DeleteEventEntity)
7   private readonly deleteRepo: Repository<DeleteEventEntity>,
8   @InjectRepository(QueryEventEntity)
9   private readonly queryRepo: Repository<QueryEventEntity>,
10 ) {}
```

```
1 constructor(
2   @InjectRepository(EventEntity)
3   private readonly eventRepo: Repository<EventEntity>,
4 ) {}
```

MANTENIMIENTO PERFECTIVO - ESTADÍSTICAS INCOMPLETAS (FALTA QUERY_EVENTS)



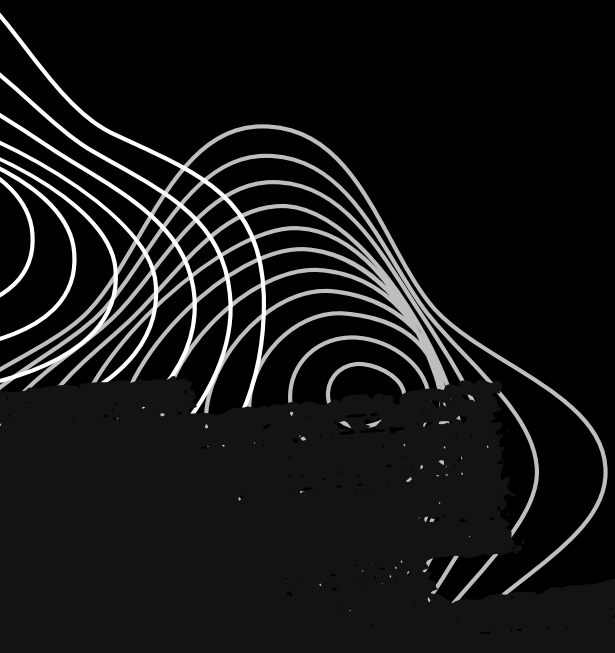
Antes

```
1 async getStats(): Promise<object> {
2   const createCount = await this.createRepo.count();
3   const updateCount = await this.updateRepo.count();
4   const deleteCount = await this.deleteRepo.count();
5   // Incidencia: query_events no se incluye en el total
6   return {
7     create: createCount,
8     update: updateCount,
9     delete: deleteCount,
10    total: createCount + updateCount + deleteCount,
11  };
12 }
```

Después

```
1 async getStats(): Promise<object> {
2   const createCount = await this.eventRepo.countBy({ action: 'CREATE' });
3   const updateCount = await this.eventRepo.countBy({ action: 'UPDATE' });
4   const deleteCount = await this.eventRepo.countBy({ action: 'DELETE' });
5   const queryCount = await this.eventRepo.countBy({ action: 'QUERY' });
6
7   return {
8     create: createCount,
9     update: updateCount,
10    delete: deleteCount,
11    query: queryCount,
12    total: createCount + updateCount + deleteCount + queryCount,
13  };
14 }
```

EL ENDPOINT /STATS REPORTABA UN TOTAL QUE EXCLUÍA TODOS LOS EVENTOS DE TIPO QUERY. UN CLIENTE QUE USARA ESE NÚMERO PARA TOMAR DECISIONES RECIBIRÍA DATOS INCORRECTOS. LA CORRECCIÓN AÑADE QUERYCOUNT AL CÁLCULO.



MANTENIMIENTO PREVENTIVO

Problemas:

- Sin límites de longitud.
- Sin validaciones numéricas.
- IDs inválidos aceptados.
- DTOs sin validaciones.
- Endpoint /health no verificaba realmente el estado de la base de datos

Solución:

- MaxLength.
- Min.
- ParseIntPipe.
- Validaciones utilizando ValidationPipe.
- Health check verifica SQLite

Se considera mantenimiento preventivo porque las mejoras implementadas buscan prevenir fallos futuros, aumentar la estabilidad del sistema y facilitar el mantenimiento posterior.

MANTENIMIENTO PREVENTIVO - MAXLENGTH EN DTO DE CREACIÓN

Antes

```
export class CreateSmartphoneDto {  
  @IsString()  
  @NotEmpty()  
  brand: string;  
  
  @IsString()  
  @NotEmpty()  
  model: string;  
  
  @IsNumber()  
  price: number;  
  
  @IsString()  
  @NotEmpty()  
  storage: string;  
}
```

Se agregaron
@MaxLength(100) en
brand,
@MaxLength(150) en
model y
@MaxLength(50) en
storage para evitar que
cadenas excesivamente
largas lleguen a la base
de datos.

Después

```
export class CreateSmartphoneDto {  
  // Antes: sin MaxLength - ahora límite 100 chars para prevenir entradas maliciosas  
  @ApiProperty({ example: 'Samsung', maxLength: 100 })  
  @IsString()  
  @NotEmpty()  
  @MaxLength(100)  
  brand!: string;  
  
  // Antes: sin MaxLength - ahora límite 150 chars  
  @ApiProperty({ example: 'Galaxy S24', maxLength: 150 })  
  @IsString()  
  @NotEmpty()  
  @MaxLength(150)  
  model!: string;  
  
  // Antes: @Min solo validado en service - ahora también validado en el DTO  
  @ApiProperty({ example: 799.99, minimum: 0.01 })  
  @IsNumber()  
  @Min(0.01)  
  price!: number;  
  
  // Antes: sin MaxLength - ahora límite 50 chars  
  @ApiProperty({ example: '128GB', maxLength: 50 })  
  @IsString()  
  @NotEmpty()  
  @MaxLength(50)  
  storage!: string;  
}
```

MANTENIMIENTO PREVENTIVO - MAXLENGTH EN DTO DE ACTUALIZACIÓN

Antes

```
export class UpdateSmartphoneDto {  
  @IsString()  
  @IsOptional()  
  brand?: string;  
  
  @IsString()  
  @IsOptional()  
  model?: string;  
  
  @IsNumber()  
  @IsOptional()  
  price?: number;  
  
  @IsString()  
  @IsOptional()  
  storage?: string;  
}
```

Se aplicaron los mismos límites de longitud que en el DTO de creación para mantener consistencia en la validación de entradas.

Después

```
export class UpdateSmartphoneDto {  
  // Antes: sin MaxLength – ahora límites consistentes con CreateSmartphoneDto  
  @ApiPropertyOptional({ example: 'Apple', maxLength: 100 })  
  @IsString()  
  @IsOptional()  
  @MaxLength(100)  
  brand?: string;  
  
  @ApiPropertyOptional({ example: 'iPhone 15', maxLength: 150 })  
  @IsString()  
  @IsOptional()  
  @MaxLength(150)  
  model?: string;  
  
  @ApiPropertyOptional({ example: 999.99, minimum: 0.01 })  
  @IsNumber()  
  @IsOptional()  
  @Min(0.01)  
  price?: number;  
  
  @ApiPropertyOptional({ example: '256GB', maxLength: 50 })  
  @IsString()  
  @IsOptional()  
  @MaxLength(50)  
  storage?: string;  
}
```


MANTENIMIENTO PREVENTIVO - PARSEINTPIPE

EN PARÁMETROS DE ID

Antes

```
@Get('/:id')
findOne(@Param('id') id: string) {
  return this.smartphonesService.findOne(Number(id));
}

@Put('/:id')
update(@Param('id') id: string, @Body() dto: UpdateSmartphoneDto) {
  return this.smartphonesService.update(Number(id), dto);
}

@Delete('/:id')
remove(@Param('id') id: string) {
  return this.smartphonesService.remove(Number(id));
}
}
```

Se reemplazó Number(id) por ParseIntPipe en los tres endpoints que reciben un ID. Ahora valores como "abc" o "-1" son rechazados con un 400 antes de llegar al servicio.

Después

```
// Antes: Number(id) sin validación, NaN llegaba al service
// Ahora: ParseIntPipe rechaza con 400 cualquier valor que no sea entero
@Get('/:id')
findOne(@Param('id', ParseIntPipe) id: number) {
  return this.smartphonesService.findOne(id);
}

@Put('/:id')
update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateSmartphoneDto) {
  return this.smartphonesService.update(id, dto);
}

@Delete('/:id')
remove(@Param('id', ParseIntPipe) id: number) {
  return this.smartphonesService.remove(id);
}
}
```

MANTENIMIENTO PREVENTIVO – MAXLENGTH EN PARÁMETROS DE EVENTOS

Antes

```
export class CreateEventDto {
  @IsString()
  @IsNotEmpty()
  source: string;

  @IsString()
  @IsNotEmpty()
  entity: string;

  @IsEnum(EventAction)
  action: EventAction;

  @IsString()
  @IsNotEmpty()
  title: string;

  @IsString()
  @IsOptional()
  description?: string;

  @IsObject()
  @IsOptional()
  payload?: Record<string, unknown>;
}
```

Se agregó
@MaxLength(100) a los
campos source y entity del
DTO de eventos para
limitar el tamaño de los
parámetros que llegan a
las consultas de base de
datos.

Después

```
export class CreateEventDto {
  // Antes: sin MaxLength – ahora límites para prevenir payloads excesivamente largos
  @ApiProperty({ example: 'epn-event-manager', maxLength: 100 })
  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  source!: string;

  @ApiProperty({ example: 'smartphone', maxLength: 100 })
  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  entity!: string;

  @ApiProperty({ enum: EventAction })
  @IsEnum(EventAction)
  action!: EventAction;

  @ApiProperty({ example: 'Smartphone creado', maxLength: 200 })
  @IsString()
  @IsNotEmpty()
  @MaxLength(200)
  title!: string;

  @ApiPropertyOptional({ example: 'Descripción del evento', maxLength: 500 })
  @IsString()
  @IsOptional()
  @MaxLength(500)
  description?: string;

  @ApiPropertyOptional({ example: { id: 1 } })
  @IsObject()
  @IsOptional()
  payload?: Record<string, unknown>;
}
```

MANTENIMIENTO PREVENTIVO - CIERRE LIMPIO DEL SERVIDOR

Antes

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );
  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

Se activó `app.enableShutdownHooks()` en `main.ts` para que la aplicación responda correctamente a señales como `SIGTERM` o `SIGINT`, permitiendo apagar el servidor de forma ordenada y reduciendo el riesgo de interrupciones bruscas.

Después

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  // Antes: sin shutdown hooks – ahora maneja SIGTERM/SIGINT limpiamente al desplegar
  app.enableShutdownHooks();
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );

  // Antes: sin documentación de API – ahora Swagger disponible en /api
  const config = new DocumentBuilder()
    .setTitle('EPN Event Manager API')
    .setDescription('API CRUD de smartphones con registro de eventos')
    .setVersion('1.0')
    .addApiKey(
      { type: 'apiKey', name: 'x-fis-eqn-key', in: 'header' },
      'x-fis-eqn-key',
    )
    .build();
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);

  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```


MANTENIMIENTO PREVENTIVO - DTOS SIN VALIDACIONES

Antes

```
1+ export class CreateTaskDto {  
2+   titulo: string;  
3+   descripcion?: string;  
4+   estado?: string;  
5+ }  
6+ |
```

Se implementó validación global de DTOs mediante ValidationPipe y decoradores de class-validator, garantizando que los datos recibidos cumplan con la estructura y reglas definidas antes de ser procesados.

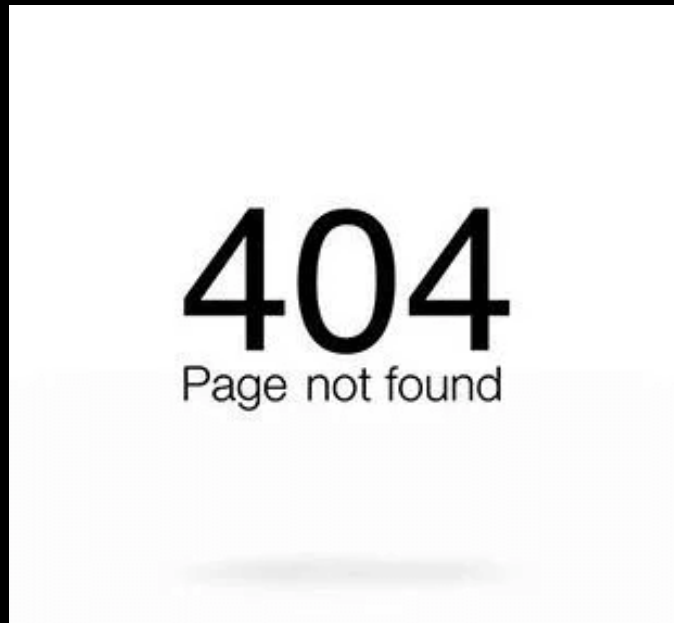
Después

```
1 import { IsString, IsOptional, IsIn } from 'class-validator';  
2  
3 export class CreateTaskDto {  
4   @IsString()  
5   titulo!: string;  
6  
7   @IsOptional()  
8   @IsString()  
9   descripcion?: string;  
10  
11  @IsOptional()  
12  @IsIn(['pendiente', 'en progreso', 'completada']) //Evita que se ingresen estados no válidos  
13  estado?: string;  
14 }  
15 |
```

```
7 async function bootstrap() {  
8   const app = await NestFactory.create<NestExpressApplication>(AppModule);  
9  
10  // Validación global de DTOs  
11  app.useGlobalPipes(  
12    new ValidationPipe({  
13      whitelist: true,  
14      forbidNonWhitelisted: true,  
15      transform: true,  
16    }),  
17  );  
18  
19  // Servir archivos estáticos desde la carpeta public  
20  app.useStaticAssets(join(__dirname, '..', 'public'));  
21  
22  await app.listen(process.env.PORT ?? 3000);  
23 }  
24 void bootstrap();
```

MANTENIMIENTO PREVENTIVO - HEALTH CHECK SUPERFICIAL

Antes



Se implementó un endpoint de Health Check con verificación real de conectividad a la base de datos, permitiendo detectar oportunamente problemas de disponibilidad y exponer el estado actual del servicio.

Después

```
1 import { Controller, Get, Inject } from '@nestjs/common';
2 import { DataSource } from 'typeorm';
3
4 @Controller('health')
5 export class HealthController {
6   constructor(@Inject(DataSource) private readonly dataSource: DataSource) {}
7
8   @Get()
9   async check() {
10     try {
11       // Verifica conectividad real a la base de datos
12       await this.dataSource.query('SELECT 1');
13       return {
14         status: 'ok',
15         database: 'connected',
16         timestamp: new Date().toISOString(),
17       };
18     } catch (error) {
19       const message = error instanceof Error ? error.message : 'Unknown error';
20       return {
21         status: 'error',
22         database: 'disconnected',
23         error: message,
24         timestamp: new Date().toISOString(),
25       };
26     }
27   }
28 }
```

PRUEBA DE VIDA

```
PS C:\Users\aaaron\Escritorio\ChauquisPro\ConstrucionYEvo\carpetaParaExamen\CHAUCA-ROJAS-AARON-ERNESTO-constswgr2\Presentation-codigo-EXAMEN\epn-event-manager\epn-event-manager> npm run start:dev
```

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Crear smartphone

Marca

Modelo

Precio

Almacenamiento

Guardar

Catalogo

ID	Marca	Modelo	Precio	Storage	
1	Xiaomi	redmi	\$300.00	256 gb	<button>Editar</button> <button>Eliminar</button>

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Smartphone creado.

Crear smartphone

Marca

Modelo

Precio

Almacenamiento

Guardar

Catalogo

ID	Marca	Modelo	Precio	Storage	
1	Xiaomi	redmi	\$300.00	256 gb	<button>Editar</button> <button>Eliminar</button>
2	Iphone	17 Pro Max	\$1500.00	256 gb	<button>Editar</button> <button>Eliminar</button>

PRUEBA DE VIDA

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Smartphone actualizado.

Crear smartphone

Marca

Modelo

Precio

Almacenamiento

Guardar

Catalogo

ID	Marca	Modelo	Precio	Storage	
1	Xiaomi	redmi	\$500.00	256 gb	Editar Eliminar
2	Iphone	17 Pro Max	\$1500.00	256 gb	Editar Eliminar

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Smartphone eliminado.

Crear smartphone

Marca

Modelo

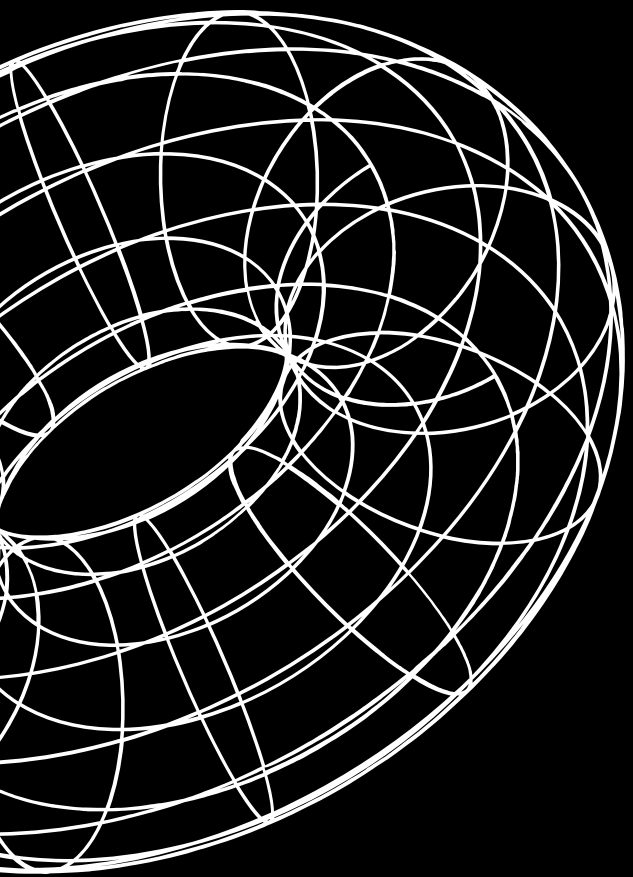
Precio

Almacenamiento

Guardar

Catalogo

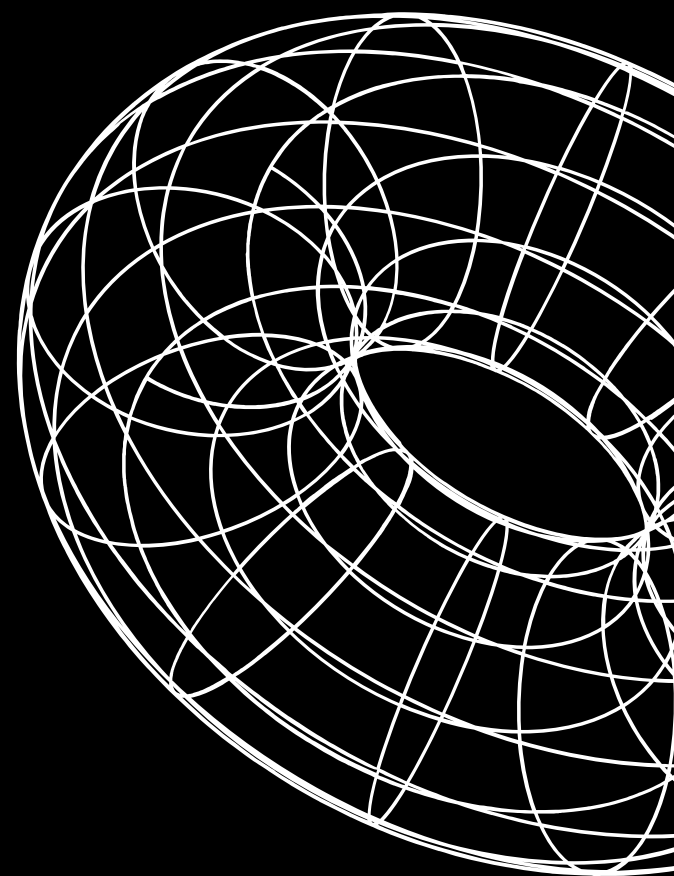
ID	Marca	Modelo	Precio	Storage	
1	Xiaomi	redmi	\$500.00	256 gb	Editar Eliminar



CONCLUSIONES

- Se identificó y clasificó la deuda técnica.
- Se aplicaron los cuatro tipos de mantenimiento.
- El sistema incrementó su seguridad y estabilidad.
- La mantenibilidad y observabilidad mejoraron significativamente.
- Se redujo el riesgo de errores futuros.

RECOMENDACIONES

- Realizar revisiones periódicas de deuda técnica.
 - Mantener validaciones, monitoreo y controles de seguridad.
 - Actualizar dependencias y tecnologías de forma continua.
 - Fortalecer la documentación técnica del sistema.
 - Aplicar mantenimiento preventivo de manera constante para reducir incidencias futuras.
- 



GRACIAS

POR SU ATENCION